

JPA

Java Persistence API

#JPA #Controller #JavaEE

Kiều Trọng Khánh

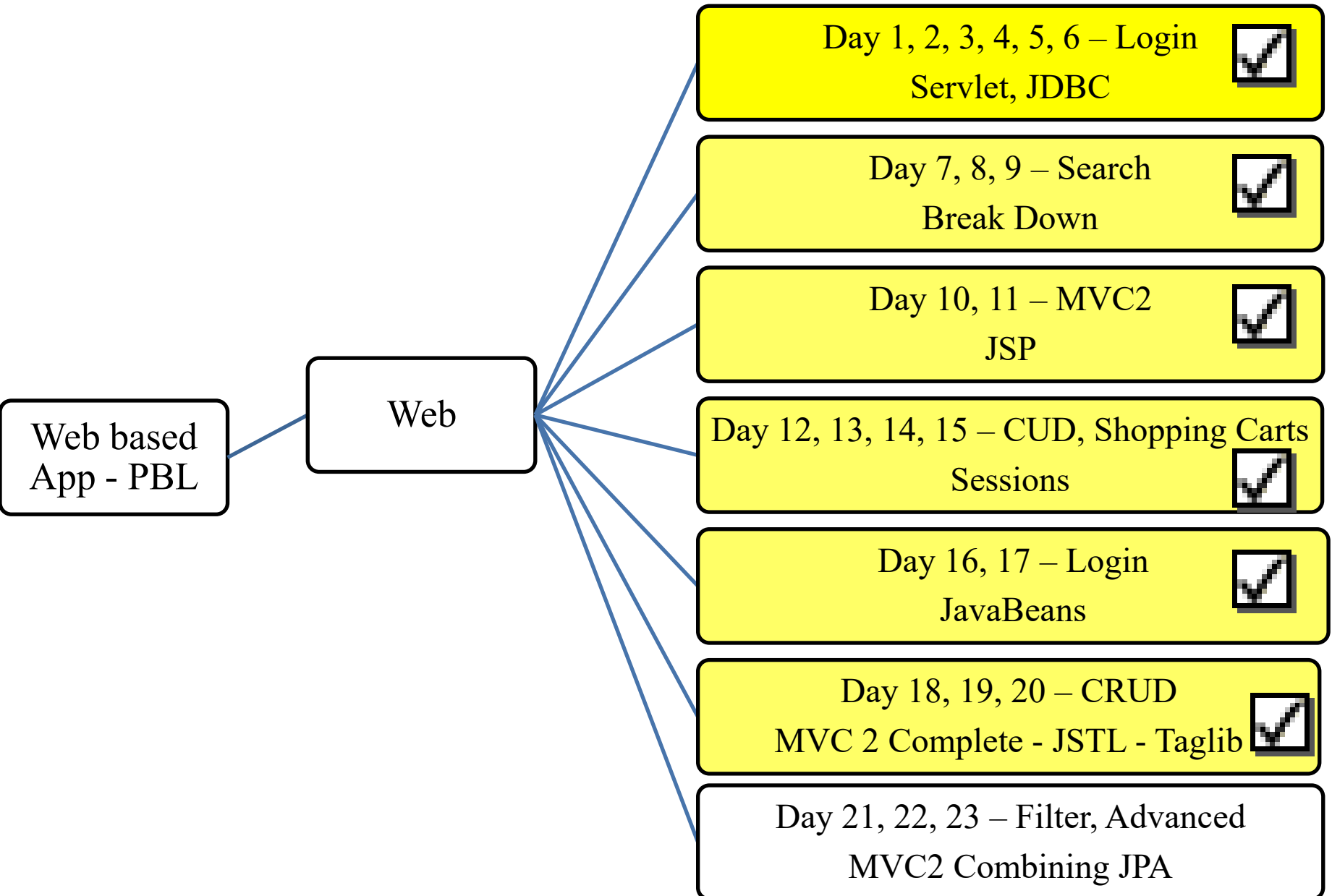
Review

- **How to build application using MVC2 Pattern using Filter as Controller?**
 - Filter
 - Filter Chain
 - Using Filter as Controller in MVC2 Design Pattern

Objectives

- **ORM**
- **JPA**
 - Definition
 - Entities
 - Entity Manager
 - JPQL
- **Apply JPA to Web Application**

Objectives

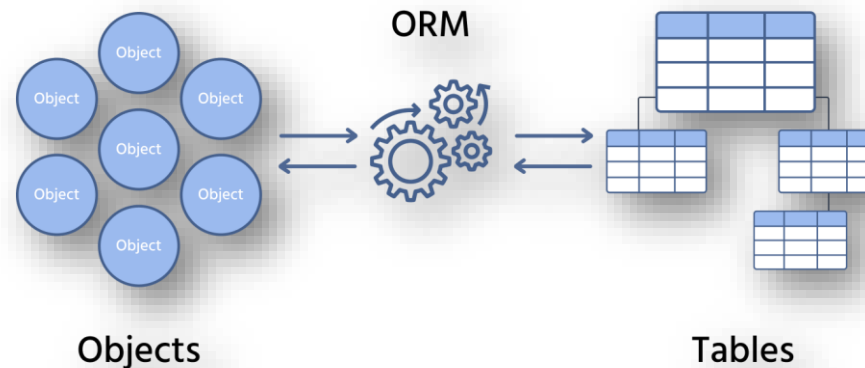


JPA

What is ORM?

ORM (Object-Relational Mapping) is a programming technique that allows developers to **map and manage data** between **object-oriented programming languages** (like **Java**) and **relational databases** (like **MySQL, PostgreSQL**)

- It abstracts the database interactions by representing tables as **Java classes**, rows as **objects**, and columns as **fields**.
- Developers can interact with the database using **object-oriented code** instead of writing raw SQL queries.



JPA

General Reasons

- **Bridges object-relational mismatch:** OO apps use classes/objects, while databases use tables/rows.
- **Increases productivity:** Reduces boilerplate JDBC/SQL code.
- **Improves maintainability:** Changes to the model layer are easier to manage.
- **Provides portability:** Code can work across different databases with minimal change.
- **Manages relationships:** Supports 1-1, 1-many, many-many relationships as object references
- Java is an OO language, and ORM is essential to translate between Java objects and relational data.
- Java ORM frameworks (like Hibernate) also integrate with JPA (Java Persistence API) – a standard for ORM in Java EE

JPA

Advantages of using ORM

Advantage	Description
Productivity	Faster development with less SQL writing
Maintainability	Centralized and consistent data model
Portability	Works across multiple databases
Lazy/Eager Loading	Controls when data is fetched
Relationship Management	Automatically maps entity relationships
Caching	Improves performance by reducing redundant DB access

JPA

Disadvantages of using ORM

Disadvantage	Description
Performance Overhead	ORM can be slower for complex queries
Learning Curve	Requires learning ORM concepts (sessions, lazy loading, etc.)
Debugging Issues	Errors might be abstracted and harder to trace
Less SQL Control	Some advanced queries are harder or inefficient via ORM
Overhead for Small Projects	May be overkill if the project has very simple data access needs

Frameworks that support ORM in Java

Framework	Description
Hibernate	Most popular ORM for Java. Implements JPA and adds advanced features.
EclipseLink	Official reference implementation of JPA. Lightweight and extensible.
Spring Data JPA	Part of the Spring ecosystem. Simplifies JPA access with repository abstraction.
TopLink	Oracle's original ORM (now evolved into EclipseLink).
Apache OpenJPA	Another open-source ORM implementation of JPA.
MyBatis	Not full ORM, but maps SQL results to objects (semi-ORM).

JPA

What is JPA?

JPA (Java Persistence API) is a Java specification (not a framework) that defines how to map Java objects (entities) to relational database tables.

- It is part of the **Java EE / Jakarta EE** platform.
- JPA provides a **set of annotations and interfaces** to manage relational data using object-oriented Java code.
- It works with ORM implementations like **Hibernate, EclipseLink, or OpenJPA.**
- **Think of JPA as the standard interface**, while Hibernate or EclipseLink are the actual **tools/libraries** that implement it.

JPA Benefit

Benefit	Description
Boilerplate reduction	Avoid writing repetitive JDBC code
Standardization	Same code works across different ORM tools (like Hibernate or EclipseLink)
Productivity	Developers can focus on business logic, not SQL
Entity management	Easy CRUD operations on Java objects
Integration	Easily works with Java EE, Spring, and other frameworks
Maintainability	Changes in database structure can be easily mapped in code

JPA

When we need to use

Situation	Reason
You have a relational database	JPA simplifies DB operations for Java apps
You need object-to-database mapping	JPA maps classes ↔ tables, fields ↔ columns
You want to avoid raw SQL	JPA supports JPQL (Java Persistence Query Language)
You build enterprise or Spring web apps	JPA is the standard for persistence
You use Hibernate or EclipseLink	These are JPA-compliant tools

JPA

How to use JPA?

✓ Step 1:

Add JPA + ORM **Dependencies**

✓ Step 2:

Create **persistence.xml**

✓ Step 3:

Create **Entity Class**

✓ Step 4:

Use **EntityManager** to interact with DB

JPA Entity

In Java, annotations are a form of metadata

- They provide data about a program but are not part of the program itself.
- An annotation is a special kind of interface preceded by `@`, and can be added to Java elements such as:
 - Classes
 - Methods
 - Variables
 - Parameters
 - Packages

JPA Entity

In **Java Persistence API (JPA)**, annotations are used to map Java objects (entities) to database tables. JPA annotations fall into several categories based on their purpose.

- Entity and Table Mapping
- Primary Key and Identity
- Column Mapping
- Relationship Mapping (Associations)
- Lifecycle Callback Annotations
- Embeddable and Inheritance

JPA

Entity and Table Mapping

Annotation	Description
@Entity	Annotate the class to indicate that it is an entity in JPA.
@Table(name = "...")	Map the entity to the corresponding table in the DB.
@MappedSuperclass	Enable the parent class to provide mapping information to its subclasses without being mapped to a table itself.
@Embeddable	Indicate that a class can be embedded in another entity as a complex attribute.
@Embedded	Embed an object marked with this into the current entity.
@Id	Primary Key.
@GeneratedValue(strategy = ...)	Auto-generate the primary key value (auto increment).
@Column(name = "...")	Map a Java class field to a column in the DB table.
@Basic	Map to column in table (without using @Column)
@Transient	The field is not mapped to column of table DB
@Temporal(...)	Combination with java.util.Date (DATE / TIME / TIMESTAMP).
@Enumerated(...)	Apply to enum (STRING or ORDINAL).

JPA

Relationship Mapping

Annotation	Description
@OneToOne	1-1 relationship between two entities.
@OneToMany	1-N relationship: one entity contains a list of other entities.
@ManyToOne	N-1 relationship: multiple entities are linked to one parent entity.
@ManyToMany	N-N relationship: mapped through a join table between two tables.
@JoinColumn	Specifies the foreign key column on the N-1 or 1-1 side.
@JoinTable	Used with @ManyToMany, defines the join table and its columns.
@MappedBy	Used on the inverse side of a bidirectional relationship.
@OrderBy	Specifies the order of the list (used in @OneToMany, etc.).
@Cascade <i>(Hibernate only)</i>	Defines cascade operations (persist, merge, remove, etc.) – only available in Hibernate, not standard in JPA.

JPA

Query Mapping

Annotation	Description
@NamedQuery	Define static JPQL queries, assign names, and reuse them.
@NamedQueries	Group multiple @NamedQuery annotations in a single class.
@NamedNativeQuery	Native SQL query.
@SqlResultSetMapping	Used to map results from native SQL to an entity or DTO.

JPA

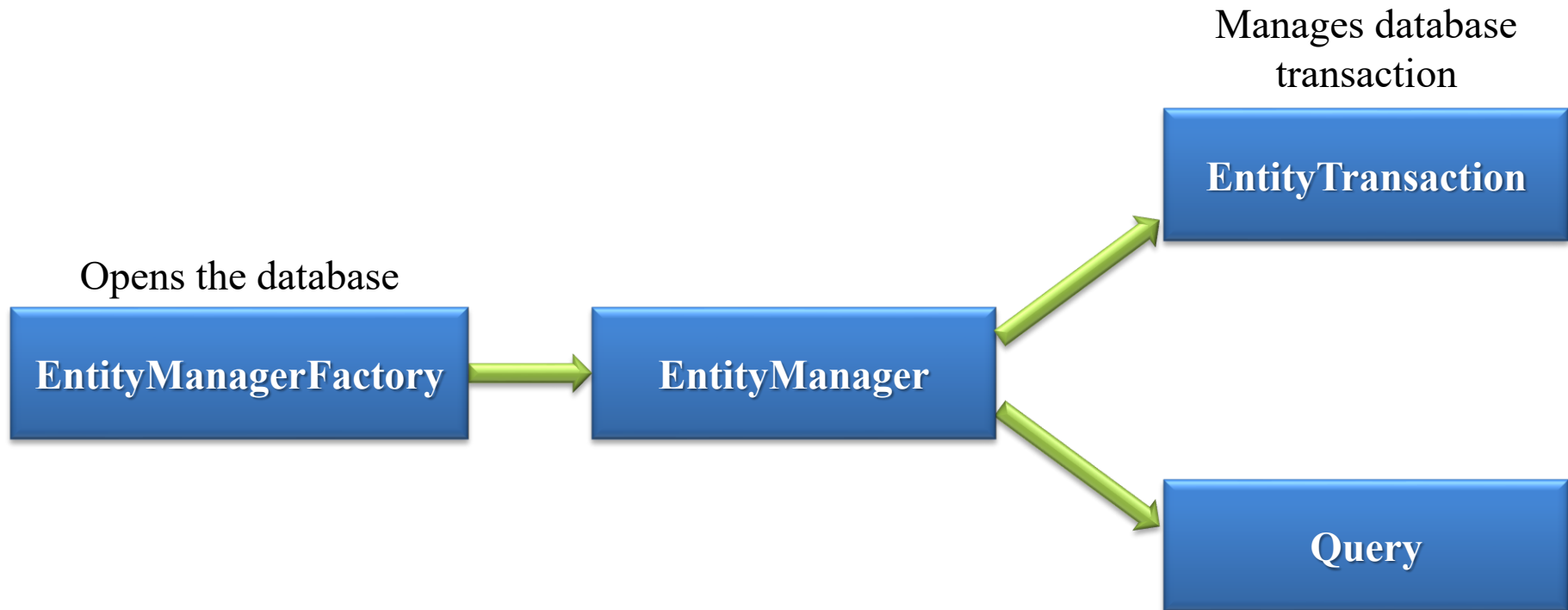
What is EntityManager?

The EntityManager is the **central interface** in JPA used to interact with the persistence context.

- **Acts** as a **bridge** between your Java objects and the database.
- **Manages** the lifecycle of entity instances and provides APIs for CRUD operations, queries, transactions, etc.
- Like a **session** that allows you to **persist, retrieve, update, and delete objects** from the DB.

JPA

How the EntityManager work?

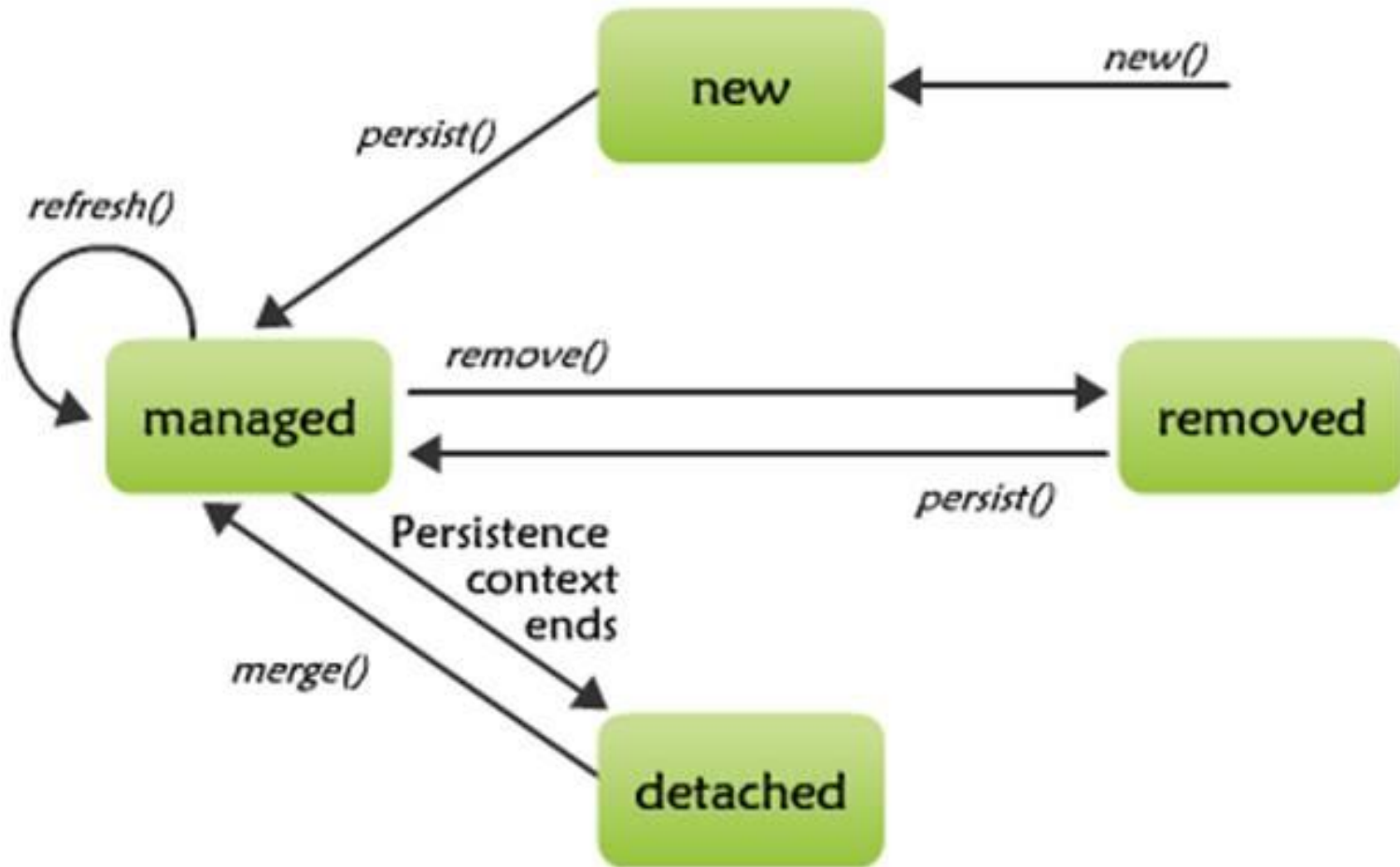


JPA EntityManager

Annotation	Descriptions
@PersistenceContext	Inject EntityManager in container env (Spring/Jakarta EE).
@PersistenceUnit	Inject EntityManagerFactory.
@ExcludeDefaultListeners, @ExcludeSuperclassListeners	Monitor listener in entity lifecycle.

JPA

Entity life cycle



Common Responsibilities of EntityManager

- Entity and Table Mapping
- Create and remove persistent entity instances
- Find entities by their primary key
- Run JPQL or native SQL queries
- Manage transactions (when not using a container)
- Synchronize entities with the database

JPA

Common Methods

Method	Description
<i>persist</i> (Object entity)	Inserts a new entity into the database
<i>find</i> (Class<T> entityClass, Object primaryKey)	Retrieves an entity by its primary key
<i>merge</i> (T entity)	Updates (merges) an entity with the current persistence context
<i>remove</i> (Object entity)	Deletes an entity from the database
<i>clear</i> ()	Clears the persistence context (detaches all entities)
<i>createQuery</i> (String jpql)	Creates a JPQL query
<i>createNativeQuery</i> (String sql)	Creates a native SQL query
<i>getTransaction</i> ()	Returns the transaction object (for manual transaction control)

JPQL – Java Persistence Query Language

JPQL

- Is used to define searches against **persistent entities independent** of the mechanism used to store those entities
- Is "**portable**", and **not constrained to any particular data store**

JPQL queries can be **declared statically in metadata**, or can be **dynamically built in code**

A **JPQL statement** may be either a **SELECT** statement, an **UPDATE** statement, or a **DELETE** statement

- Same as the SQL Syntax
- **Support dot navigation (.)**

JPA

JPQL – From Clause

- FROM clause – mandatory –
 - contains the name of an **abstract schema** along with a **variable** declaration is **unique** to JPQL
 - The query **fetches** the **data** from the tables mapped to the abstract schema, and **save** it in this variable
- JPQL Variable
 - Used in the FROM clause is not case-sensitive
 - Must adhere to the following rules:
 - **First character** must be either **an alphabet**, a **dollar** (\$) sign, or an **underscore** (_).
 - **Second and subsequent characters** must be **digit**, an **alphabet**, a **dollar** (\$) sign, or an **underscore** (_).
 - Must not contain the reserved EJB QL keywords such as **AND**, **AS**, **BETWEEN**, **DISTINCT**, **EMPTY**, **FALSE**, **FROM**, **IN**, **IS**, **LIKE**, **MEMBER**, **NOT**, **NULL**, **OBJECT**, **OF**, **OR**, **SELECT**, **TRUE**, **UNKNOWN**, **AND WHERE**.

JPA

JPQL – Where Clause

- JPQL Input Parameters
 - Either **positional** or **named parameters** may be used. Positional and named parameters may not be mixed in a single query.
 - Can only be used in the **WHERE** clause or **HAVING** clause of a query.
- **Position** Parameters
 - Input parameters are designated by the **question mark (?)** prefix followed by an integer.
 - **Ex:** a.username = ?1
- **Named** Parameters
 - Is an identifier that is prefixed by the **":"** symbol
 - Are **case sensitive**. **Ex:** a.username = :username

JPA

JPQL – Common Types

- **Static Query (Named Query)**
 - Predefined in entity classes using **@NamedQuery** annotation.
 - **Loaded** into memory at **deployment time**.
 - **Reusable** across multiple requests with parameters with **No syntax check at runtime**
 - Used with: **createNamedQuery()**
- **Dynamic Query**
 - Created at **runtime** when the method is called.
 - Parsed and validated **during execution**
 - More flexible but slightly slower
 - Used with: **createQuery()**
- **Native Query**
 - Uses **raw SQL**, written exactly like in the database
 - Useful when you **need advanced SQL features**
 - Created using: **createNativeQuery()**

JPA

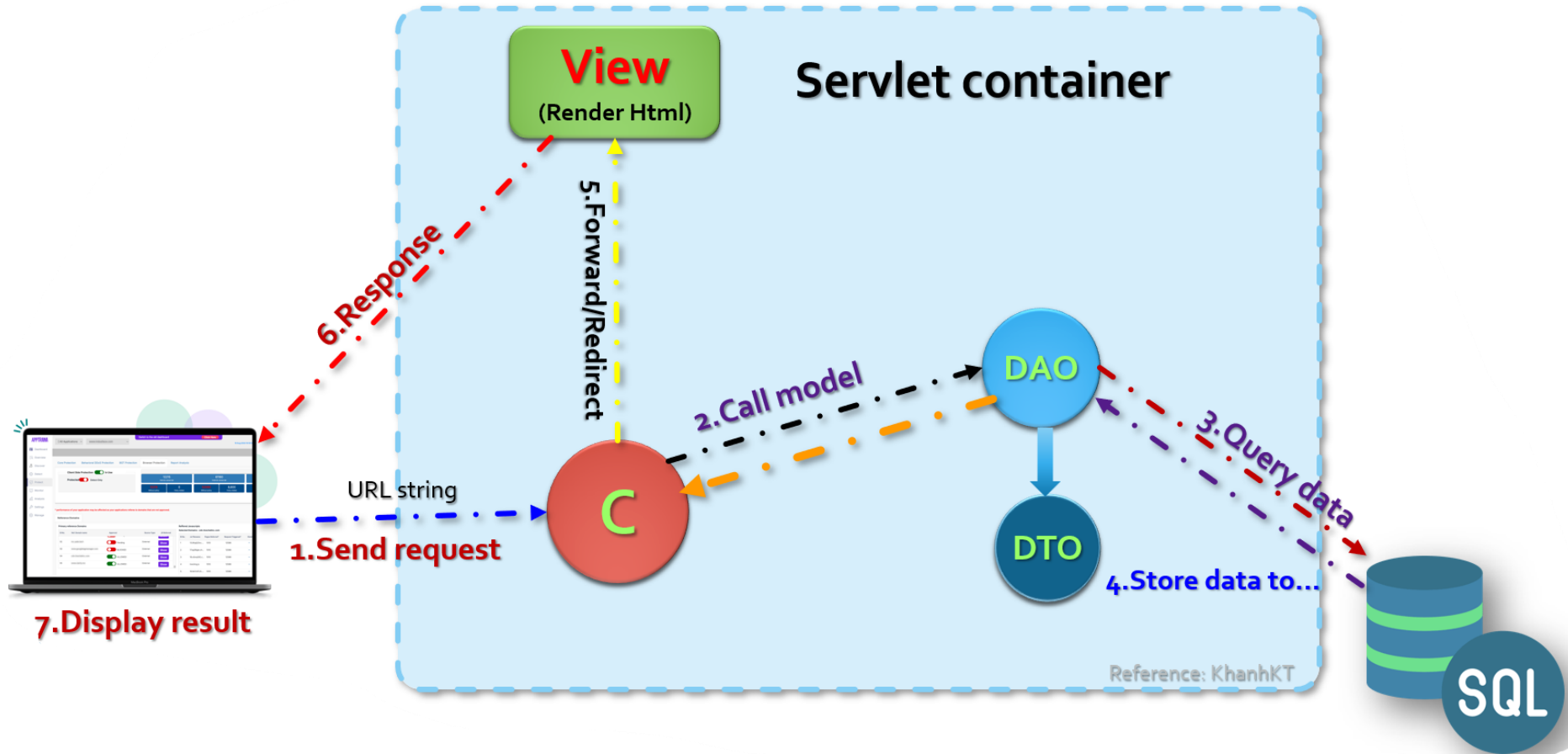
JPQL – Get Result Method

Method	Description
List getResultList()	Return entities collection . The null value is returned when no entities found.
Object getSingleResult()	Return the single entity (should cast to entity type). The NoResultException will be thrown when no entities found The NonUniqueResultException will be thrown when more than one entities found.
Query setParameter(String name, Object value)	Pass a specific parameter to query

Summary

JPQL vs. (DAO, DTO, ...)

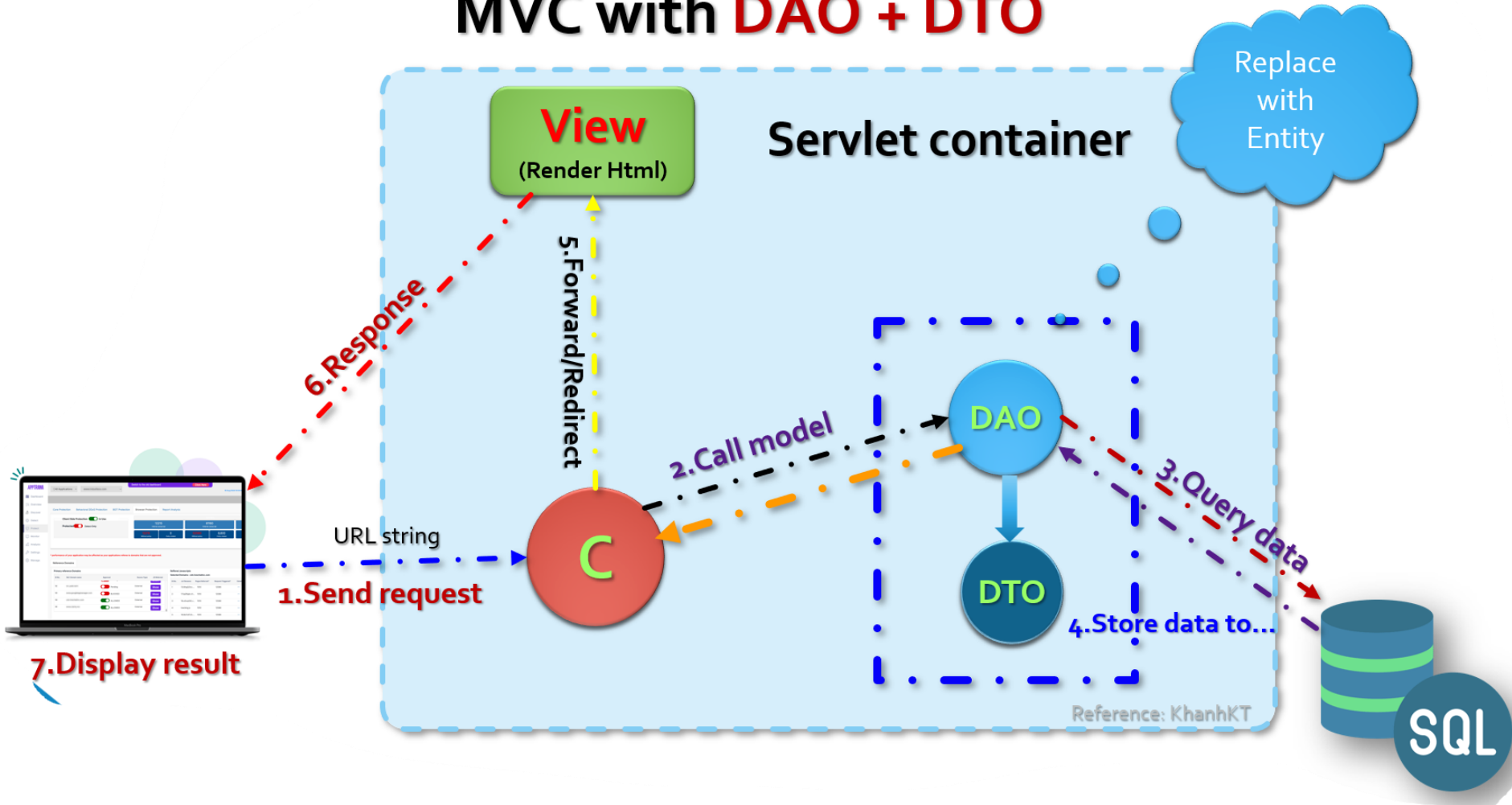
MVC with **DAO + DTO**



Summary

JPA vs. (DAO, DTO, ...)

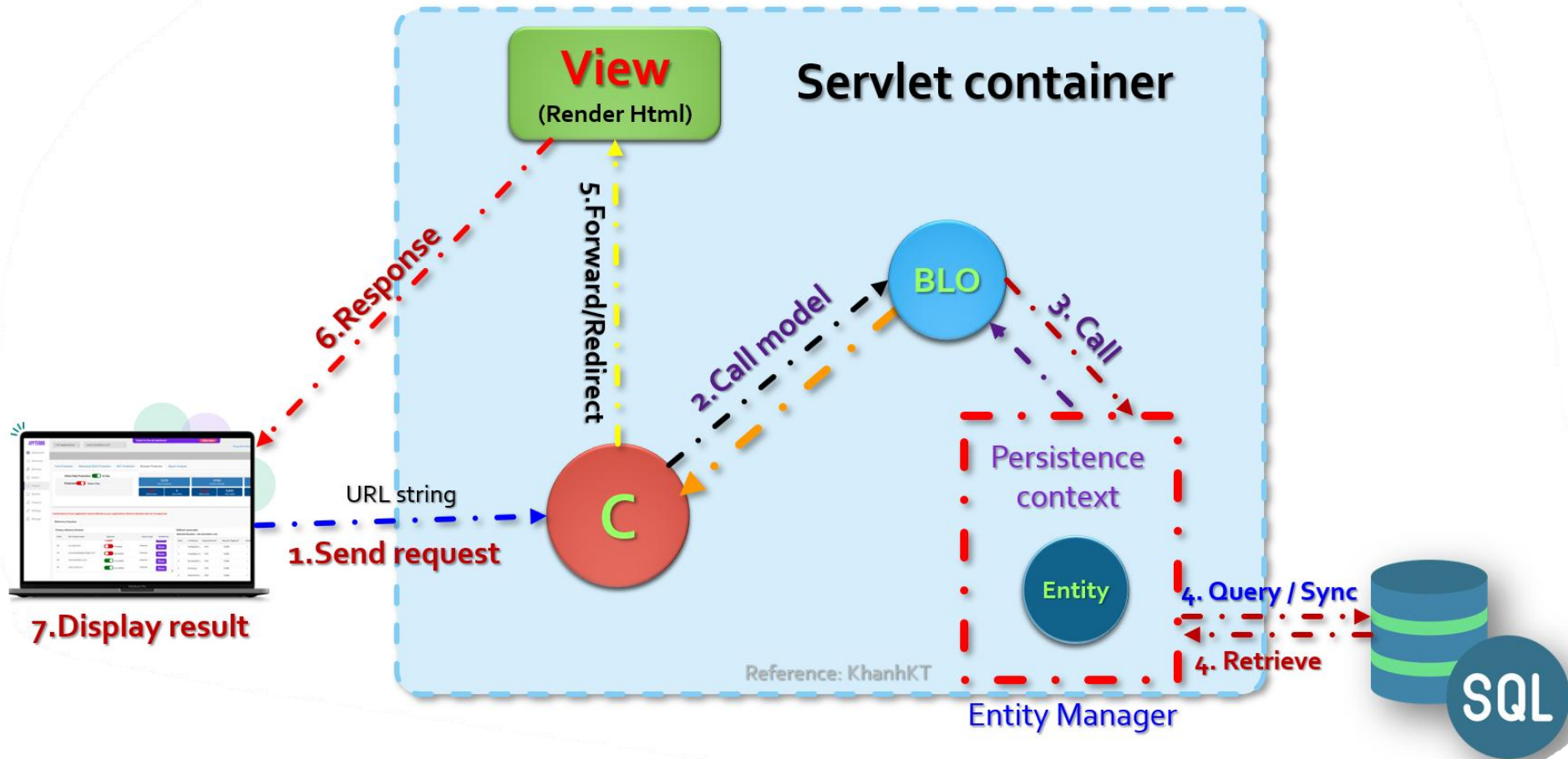
MVC with **DAO + DTO**



Summary

JPA vs. (DAO, DTO, ...)

MVC with **Entity**



Apply JPA to Web Application DataSource

New Connection Wizard

Customize Connection

Driver Name:

Host: Port:

Database:

Instance Name:

User Name:

Password:

☒ Remember password

JDBC URL:

New Connection Wizard

Choose Database Schema

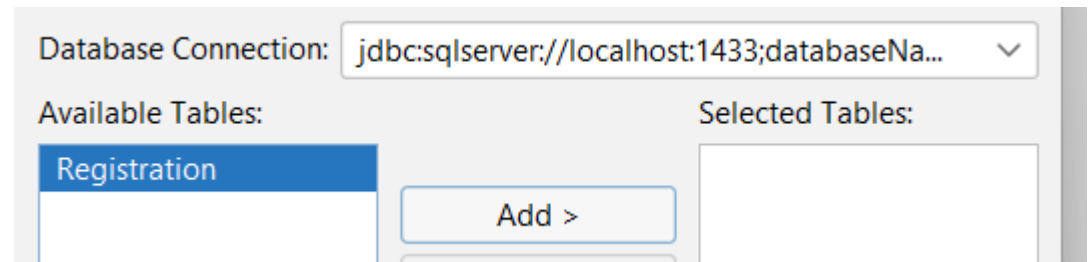
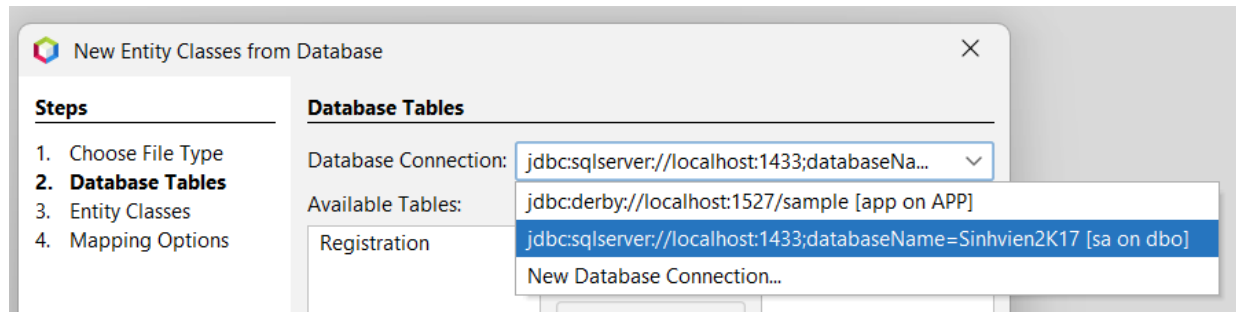
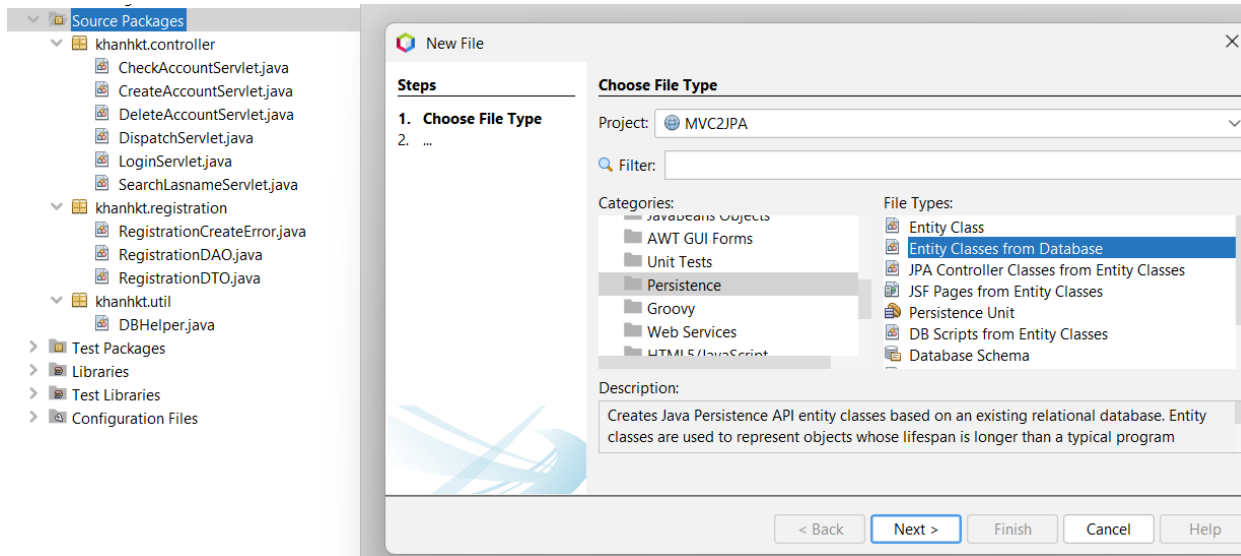
For each database connection, the Services window only displays objects from one database schema.
Select the schema of the tables to be displayed.

Select schema:

Input connection name:

Apply JPA to Web Application

Create Entities



Apply JPA to Web Application

Create Entities

New Entity Classes from Database

Steps

1. Choose File Type
- 2. Database Tables**
3. Entity Classes
4. Mapping Options

Database Tables

Database Connection: jdbc:sqlserver://localhost:1433;databaseNa...

Available Tables:

Selected Tables:

Registration

Add >

< Remove

Add All >>

<< Remove All

Any

< Back Next > Finish

New Entity Classes from Database

Entity Classes

Specify the names and the location of the entity classes.

Class Names:

Database Table	Class Name	Generation Type
Registration	Registration	New

Project: MVC2JPA

Location: Source Packages

Package: khanhkt.controller

☒ Generate Named Query Annotations for Persistent Fields

☒ Generate JAXB Annotations

☐ Generate MappedSuperclasses instead of Entities

☒ Create Persistence Unit

< Back Next > Finish Cancel Help

Apply JPA to Web Application

Create Entities


New Entity Classes from Database

Steps

1. Choose File Type
2. Database Tables
3. Entity Classes
- 4. Mapping Options**

Mapping Options

Specify the default mapping options.

Association Fetch:

Collection Type:

☒ Fully Qualified Database Table Names

☒ Attributes for Regenerating Tables

☒ Use Column Names in Relationships

☐ Use Defaults if Possible

☐ Generate Fields for Unresolved Relationships

Apply JPA to Web Application

Create Entities

Registration.java
Source
History

```

4  /*
5  package khanhkt.registration;
6
7  import java.io.Serializable;
8  import javax.persistence.Basic;
9  import javax.persistence.Column;
10 import javax.persistence.Entity;
11 import javax.persistence.Id;
12 import javax.persistence.NamedQueries;
13 import javax.persistence.NamedQuery;
14 import javax.persistence.Table;
15 import javax.xml.bind.annotation.XmlRootElement;
16
17 /**...4 lines */
18
19 @Entity
20 @Table(name = "Registration", catalog = "Sinhvien2K17", schema = "dbo")
21 @XmlRootElement
22 @NamedQueries({
23     @NamedQuery(name = "Registration.findAll",
24         query = "SELECT r FROM Registration r"),
25     @NamedQuery(name = "Registration.findByUsername",
26         query = "SELECT r FROM Registration r WHERE r.username = :username"),
27     @NamedQuery(name = "Registration.findByPassword",
28         query = "SELECT r FROM Registration r WHERE r.password = :password"),
29     @NamedQuery(name = "Registration.findByLastname",
30         query = "SELECT r FROM Registration r WHERE r.lastname = :lastname"),
31     @NamedQuery(name = "Registration.findByIsAdmin",
32         query = "SELECT r FROM Registration r WHERE r.isAdmin = :isAdmin")})
33
34 public class Registration implements Serializable {
35
36     @Override
37     public int hashCode() {
38         int hash = 0;
39         hash += (username != null ? username.hashCode() : 0);
40         return hash;
41     }
42
43     @Override
44     public boolean equals(Object object) {
45         // TODO: Warning - this method won't work in the case the id fields are not set
46         if (!(object instanceof Registration)) {
47             return false;
48         }
49         Registration other = (Registration) object;
50         if ((this.username == null && other.username != null) || (this.username != null &&
51             !this.username.equals(other.username))) {
52             return false;
53         }
54         return true;
55     }
56
57     @Override
58     public String toString() {
59         return "khanhkt.registration.Registration[ username=" + username + " ]";
60     }
61 }

```

```

public class Registration implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    @Basic(optional = false)
    @Column(name = "username", nullable = false, length = 20)
    private String username;
    @Basic(optional = false)
    @Column(name = "password", nullable = false, length = 30)
    private String password;
    @Basic(optional = false)
    @Column(name = "lastname", nullable = false, length = 50)
    private String lastname;
    @Basic(optional = false)
    @Column(name = "isAdmin", nullable = false)
    private boolean isAdmin;

    public Registration() { ...2 lines }

    public Registration(String username) { ...3 lines }

    public Registration(String username, String password, String lastname, boolean
    isAdmin) { ...4 lines }

    public String getUsername() { ...3 lines }

    public String getPassword() { ...3 lines }

    public String getLastname() { ...3 lines }

    public boolean isAdmin() { ...3 lines }

}

```

Apply JPA to Web Application

Create Entities

Configuration Files

- MANIFEST.MF
- context.xml
- persistence.xml
- web.xml

Libraries

- sqljdbc4.jar
- jakarta.servlet.jsp.jstl-2.0.0.jar
- jakarta.servlet.jsp.jstl-api-2.0.0.jar
- EclipseLink (JPA 2.1) - org.eclipse.per
- EclipseLink (JPA 2.1) - org.eclipse.per
- EclipseLink (JPA 2.1) - org.eclipse.per
- EclipseLink (JPA 2.1) - org.eclipse.per
- EclipseLink (JPA 2.1) - org.eclipse.per
- EclipseLink (JPA 2.1) - org.eclipse.per
- EclipseLink (JPA 2.1) - javax.persisten
- JDK 1.8 (Default)
- Apache Tomcat 10.0.27

Registration.java × persistence.xml ×

Design Source History

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <persistence version="2.1" xmlns="http://xmlns.jcp.org/xml/ns/persistence" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3   <persistence-unit name="MVC2JPAPU" transaction-type="RESOURCE_LOCAL">
4     <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
5     <class>khanhkt.registration.Registration</class>
6     <exclude-unlisted-classes>false</exclude-unlisted-classes>
7     <properties>
8       <property name="javax.persistence.jdbc.url" value="jdbc:sqlserver://localhost:1433;databaseName=Sinhvien2K17"/>
9       <property name="javax.persistence.jdbc.user" value="sa"/>
10      <property name="javax.persistence.jdbc.driver" value="com.microsoft.sqlserver.jdbc.SQLServerDriver"/>
11      <property name="javax.persistence.jdbc.password" value="trongkhanh"/>
12    </properties>
13  </persistence-unit>
14 </persistence>
15
```

Apply JPA to Web Application Consume

```
/**...4 lines */
```

```
public class RegistrationBLO
```

```
@Override
```

```
public
```

```
    t
```

```
    }
```

```
@Override
```

```
public
```

Navigate

Show Javadoc

Find Usages

Call Hierarchy

Insert Code...

Alt+F1

Alt+F7

Alt+Insert

Generate

Constructor...

Logger...

toString()...

Override Method...

Add Property...

Use Entity Manager...

Use Database...

Call Web Service Operation...

RegistrationBLO.java

Source

History

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("MVC2JPAPU");
```

```
public RegistrationBLO() { ...2 lines }
```

```
@Override
```

```
public void close() {
```

```
    if (this.emf != null) {
```

```
        if (this.emf.isOpen()) {
```

```
            this.emf.close();
```

```
        }
```

```
    }
```

```
}
```

Apply JPA to Web Application

Consume

```

RegistrationBLO.java x
Source History
34 }
35
36 public Registration checkLogin(String username, String password) {
37     EntityManager em = this.emf.createEntityManager();
38     Registration result = null;
39     String jpql = "Select r "
40                 + "From Registration r "
41                 + "Where r.username = :username "
42                 + "And r.password = :password";
43     try {
44         Query query = em.createQuery(jpql);
45         query.setParameter("username", username);
46         query.setParameter("password", password);
47         result = (Registration) query.getSingleResult();
48     } finally {
49         em.close();
50     }
51
52     return result;
53 }
  
```

Apply JPA to Web Application

Consume

RegistrationBLO.java
Source
History

```

54
55 public List<Registration> searchLastname(String searchValue) {
56     EntityManager em = this.emf.createEntityManager();
57     List<Registration> result = null;
58     String jpql = "Select r "
59                 + "From Registration r "
60                 + "Where r.lastname Like :lastname";
61     try {
62         Query query = em.createQuery(jpql);
63         query.setParameter("lastname", "%" + searchValue + "%");
64
65         result = query.getResultList();
66     } finally {
67         em.close();
68     }
69
70     return result;
71 }

```

Apply JPA to Web Application

Consume

RegistrationBLO.java
Source
History

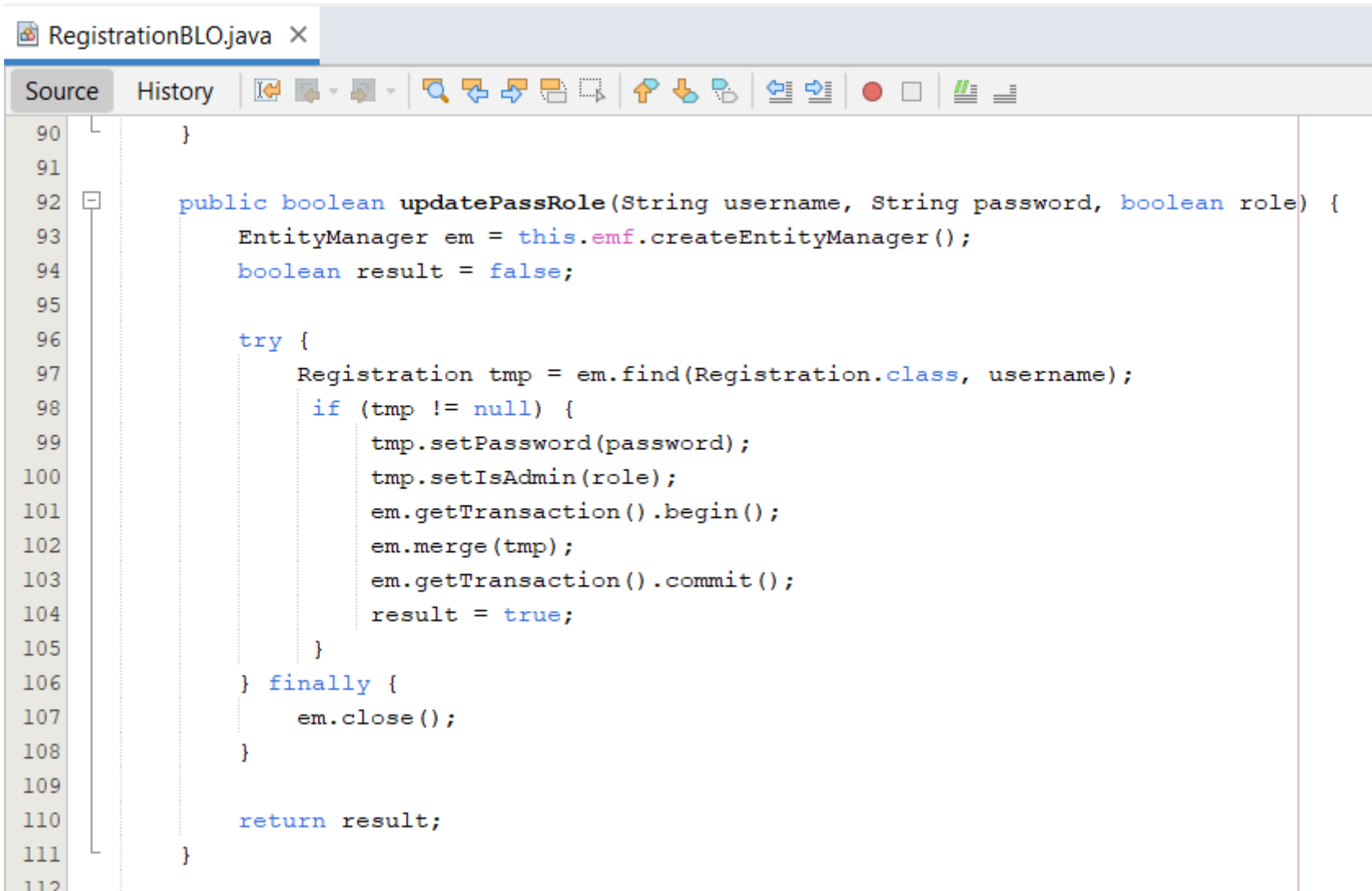
```

72
73 public boolean delete(String username) {
74     EntityManager em = this.emf.createEntityManager();
75     boolean result = false;
76
77     try {
78         Registration tmp = em.find(Registration.class, username);
79         if (tmp != null) {
80             em.getTransaction().begin();
81             em.remove(tmp);
82             em.getTransaction().commit();
83             result = true;
84         }
85     } finally {
86         em.close();
87     }
88
89     return result;
90 }

```

Apply JPA to Web Application

Consume



```
RegistrationBLO.java x
Source History
90     }
91
92     public boolean updatePassRole(String username, String password, boolean role) {
93         EntityManager em = this.emf.createEntityManager();
94         boolean result = false;
95
96         try {
97             Registration tmp = em.find(Registration.class, username);
98             if (tmp != null) {
99                 tmp.setPassword(password);
100                 tmp.setIsAdmin(role);
101                 em.getTransaction().begin();
102                 em.merge(tmp);
103                 em.getTransaction().commit();
104                 result = true;
105             }
106         } finally {
107             em.close();
108         }
109
110         return result;
111     }
112 }
```

Apply JPA to Web Application

Consume

RegistrationBLO.java ×

Source

History

```
181
182     @Override
183     public boolean create(Registration x) {
184         EntityManager em = this.emf.createEntityManager();
185         boolean result = false;
186
187         try {
188             Registration tmp = em.find(Registration.class, x.getUsername());
189             if (tmp == null) {
190                 this.persist(x);
191                 result = true;
192             }
193         } finally {
194             em.close();
195         }
196
197         return result;
198     }
```

Apply JPA to Web Application

Consume

RegistrationBLO.java X

Source

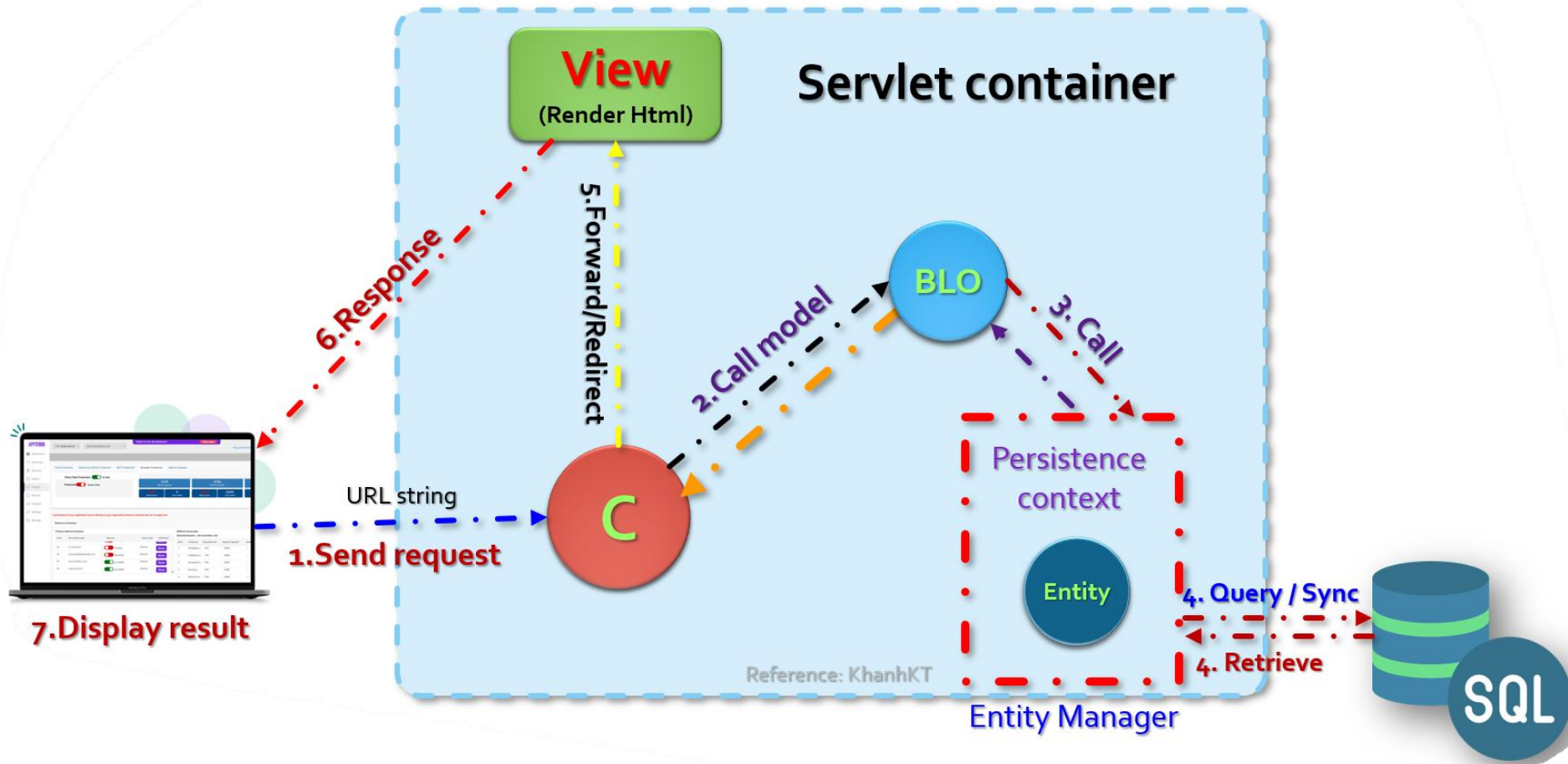
History



```
23 public void persist(Object object) {
24     EntityManager em = emf.createEntityManager();
25     try {
26         em.getTransaction().begin();
27         em.persist(object);
28         em.getTransaction().commit();
29     } catch (Exception e) {
30         em.getTransaction().rollback();
31     } finally {
32         em.close();
33     }
34 }
```

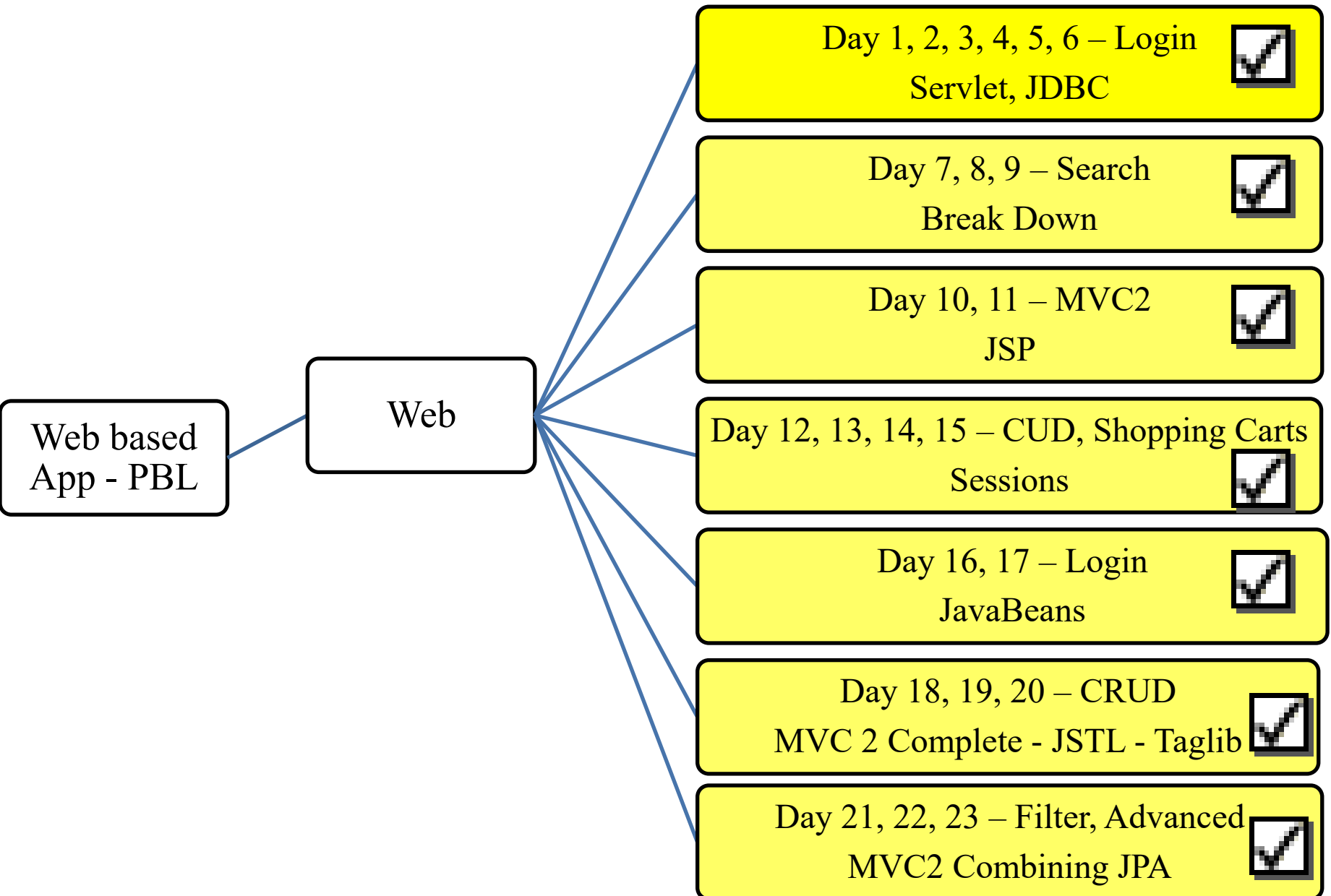
Summary

MVC with Entity



Q&A

Summary



Next Days

- Practice with exercises
- Practical Test
- Presentations

Good luck to you!!!